

General Overview

The Extended Kalman filter architecture we are exploiting the data obtained from the sensors: BNO055 (body acceleration, and angular velocity) MRPLS for pressure (from which it is possible to derive an altitude using an atmosphere model), and a GPS (u-blox MAX-M-10S).

The goal of this system is to filter out the noise of the sensors and fuse them to better estimate the current altitude of the rocket. Using the sensor information we will estimate this value through mathematical computations: calculating the rocket orientation and integrating the acceleration.

The model will be constructed based on acceleration measurements provided by the BNO055 sensor, which will be converted into absolute acceleration using the angular velocity data—also supplied by the same device. These values will then be integrated to obtain velocity and position along the vertical axis. The reference directions for the first two axes will be established during a calibration phase, performed while the rocket is at rest, by exploiting the BNO055's gravity reading. To improve accuracy, an Extended Kalman Filter (EKF) will be implemented, correcting the 'up' position using altitude data provided by the MRPLS pressure sensor and the GPS.

The EKF aims to estimate the state of the variables of interest—such as position, velocity and orientation (represented mathematically by the quaternion)—by following a two-step recursive procedure.

First, based on a physical model, the EKF predicts the next state; this phase is known as the **prediction step**, during which it also estimates the associated uncertainty.

Then, in the **update step**, the filter incorporates the latest measurements to correct the predicted state and refine its uncertainty.

Depending on the relative uncertainties of the prediction and the measurement, the EKF assigns more weight either to the model-based prediction or to the measurement update, striking a dynamic balance between both sources of information.

Matrices and their Values

The mathematical formulation of the Extended Kalman Filter (EKF) is articulated through a series of matrix-based algebraic equations, which provide a compact and systematic representation of the system dynamics, uncertainties, and sensor measurements.

The state vector of the proposed model comprises the altitude and vertical velocity and the four components of a quaternion representing the rotation from the IMU's local reference frame to the absolute frame. This results in a 6-dimensional state vector, denoted by $\mathbf{x}$. The physical model equations are described by:

$$\begin{aligned}x_k &= f(x_{k-1}) + w_k \\z_k &= h(x_k) + v_k\end{aligned}$$

where f is the model's function, w_k represents process noise (e.g., a wind gust), z_k is the sensor measurement and v_k is zero-mean white noise affecting the measurement.

In the prediction step, the EKF estimates the next state and its associated uncertainty:

$$\begin{aligned}\hat{x}_k &= f(\hat{x}_{k-1}, u_k) \\P_k &= F_{k-1} P_{k-1} F_{k-1}^T + Q_{k-1}\end{aligned}$$

Here, $\hat{x}$, is the predicted state estimate, P_k is the predicted covariance matrix, F is the jacobian matrix of the system model and Q_k denotes the process noise covariance, which quantifies the model's deviation from reality.

The update step is then expressed as:

$$\begin{aligned}G_k &= P_k H_k^T (H_k P_k H_k^T + R)^{-1} \\ \hat{x}_k &\leftarrow \hat{x}_k + G_k (z_k - h(\hat{x}_k)) \\ P_k &\leftarrow (I - G_k H_k) P_k\end{aligned}$$

Here, G_k is the Kalman gain matrix, which balances the influence of the model versus the measurements. A higher gain implies greater trust in the measurements. R_k is the measurement noise covariance matrix, reflecting the variability of sensor readings. Finally, it updates the uncertainty matrix P_k .

For this project, the EKF is implemented using the **TinyEKF** library in C++. To adapt the library to this model, it is necessary to provide the matrices F_k , H_k , Q_k and R_k . The state vector x_k and the measurement vector z_k are defined as:

$$\begin{aligned}x &= [z \quad v_z \quad q_w \quad q_x \quad q_y \quad q_z] \\ \mathbf{z} &= [a_x \quad a_y \quad a_z \quad \omega_x \quad \omega_y \quad \omega_z \quad h_p \quad z_{GPS}]\end{aligned}$$

The matrix F_k is initialized as the identity matrix and then computed numerically, since deriving it analytically would be overly complex. The measurement matrix H matrix is the jacobian matrix of the measurement vector z with respect to the state vector x . It transforms absolute accelerations into relative ones according to the current orientation. Q is the covariance matrix of the model and R is the sensor covariance matrix, for which, we have set the following values according to the best results based in sensor simulation:

- Altitude: $1e-4 \text{ (m}^2\text{)}$.
- Vertical velocity: $1e-4 \text{ (m}^2\text{/s}^2\text{)}$.
- Quaternion: $1e-8$.

And for the measurement noise covariance matrix R (sensors):

- Accelerometer measurement noise: $1e-3 \text{ (m}^2\text{/s}^4\text{)}$.

- Gyroscope measurement noise: $1\text{e-}6 \text{ (rad}^2/\text{s}^2\text{)}$.
- GPS altitude measurement noise: $1 \text{ (m}^2\text{)}$.
- Pressure-derived altitude, varying in the range 1–300 depending on velocity (m^2).

Both R and Q have been set as diagonal because it has been supposed that there is no correlation between any of the variables, which simplifies the model, but it eases the implementation.

The matrix Q characterizes the uncertainty associated with the physical model—essentially, how much we expect the real system to deviate from the predictions given by our dynamic equations. In this work, the elements of Q have been set to relatively small values. This choice reflects the initial assumption that the physical model used to describe the system dynamics is highly accurate and reliable under nominal conditions.

By assigning low variances we are indicating to the filter that the system is expected to evolve in a predictable manner and that any deviations from this behavior are minimal. This conservative initialization is common in early stages of filter design, particularly when the system has not yet undergone real-world testing. It serves as a baseline configuration which can later be refined.

More accurate values for Q should be determined empirically—ideally after performing a real launch or a controlled experimental campaign. During such tests, one can compare the predicted states against ground truth measurements and compute the residuals over time, allowing for a data-driven estimation of the actual model uncertainties.

To define the values of the measurement noise covariance matrix R , we recorded data from the IMU while it was at rest and computed the variance of each sensor axis with respect to its mean. These variances, assumed uncorrelated across axes, were used as the diagonal values of R . This approach gives a realistic initial estimate of the sensor noise based on real data, which helps the filter behave reliably in early tests.

Selection of Δt

Estimation of the error according to a discrete model, based on a Taylor Series Expansion after 20 seconds of flight. It has been taken into account the different phases of the flight (ignition, ~constant thrust, engine off, free flight):

Simplifying the computation of the position we are making with the EKF, to the following quadratic expression:

$$\hat{x}_k = \hat{x}_{k-1} + \hat{v}_k \Delta t + \frac{a_k}{2} \Delta t^2$$

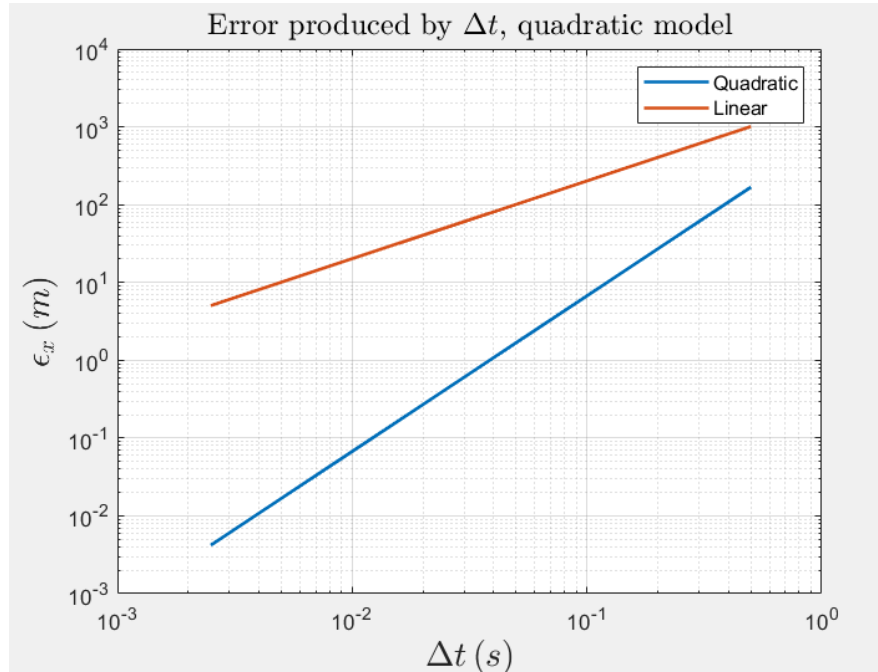
The error will be:

$$\epsilon_x = x_k - \hat{x}_k = \frac{a'(\xi)}{6} \Delta t^3, \quad \xi \in [t_{0,i}, t_{f,i}]$$

where ξ belongs to the time instant between the initial and final time instant of the phase i . For each phase, it has been considered the following values for the derivative of the acceleration, according to the simulation done by the GNC department:

- $a' = [10^4 \ 20 \ 10^4 \ 1] \text{ m/s}^3$
- $t_f = [0.2 \ 1 \ 2.2 \ 20] \text{ seconds}$

Then, plotting ϵ_x versus Δt , taking into account each of the phases, we obtain:



The frequency of the sensor is 100 Hz, so the maximum value for Δt is 0.01, therefore, with no correction of the position, after 20 seconds of flight, and with a linear model, the error is around 20 meters if $\Delta t = 0.01$ s. Taking the advantage of a GPS, or an altimeter, the error for the position can be reduced up to a constant 5-20 meters depending on the precision of the GPS and IMU.

Implementation Details

- Sampling rate and timing considerations
- Code snippets or pseudocode of the implementation
- Calibration
- Computational Performance on the hardware

External References

[Github Repository](#)

[TinyEKF Documentation](#)

[TinyEKF Repository](#)